

Módulo II: Framework Laravel

¿Qué crea Laravel? (Estructura de carpetas)

Cuando se ejecuta `composer create-project`, Laravel genera una estructura de carpetas. Se hará una breve descripción una por una

Carpeta	Función
app/	Contiene el núcleo de la aplicación. Aquí van los Modelos, Controladores, y la lógica de negocio.
bootstrap/	Archivos que "arrancan" el framework. No se modifica casi nunca.
config/	Todos los archivos de configuración de Laravel (base de datos, mail, cache, etc.).
database/	Migraciones, seeders y factories (para poblar la BD).
public/	Punto de entrada de la aplicación (<code>index.php</code>). Aquí van CSS, JS, imágenes.
resources/	Vistas (Blade), archivos CSS/JS sin compilar.
routes/	Definición de todas las rutas (URLs) de la aplicación.
storage/	Archivos generados por Laravel (logs, caché, sesiones).
vendor/	TODAS las dependencias instaladas por Composer. No se toca.
.env	Variables de entorno (configuración específica del entorno).

Patrón MVC (Modelo-Vista-Controlador)

MVC es un patrón de arquitectura de software que separa la aplicación en tres componentes:

1. **Modelo (M)**: Representa los datos y la lógica de negocio. Habla con la base de datos. En Laravel, los modelos están en `app/Models/`.
2. **Vista (V)**: Es lo que el usuario ve. La interfaz. En Laravel, las vistas están en `resources/views/` y usan el motor **Blade**.
3. **Controlador (C)**: Recibe las peticiones del usuario, habla con el Modelo para obtener datos, y decide qué Vista mostrar. En Laravel, los controladores están en `app/Http/Controllers/`.

Analogía para entender MVC:

Imaginar un restaurante:

- **Modelo** = La cocina (donde se prepara la comida - los datos).
- **Vista** = El menú y la presentación del plato (lo que ve el cliente).
- **Controlador** = El mesero (recibe el pedido, va a la cocina, trae la comida).

¿Por qué es útil?

- Separa responsabilidades.
- Facilita el mantenimiento.
- Permite que diseñadores trabajen en las Vistas y programadores en los Modelos/Controladores.

Flujo de una petición en Laravel

```

Usuario → Ruta (routes/web.php) → Controlador → Modelo → BD
                                     ↓
                               ← Respuesta ← Vista ← Controlador ←
  
```

¿Qué pasa cuando un usuario escribe una URL en el navegador?

1. Usuario escribe: <http://localhost:8000/usuarios>
2. El archivo public/index.php recibe la petición (punto de entrada)
3. Laravel busca en routes/web.php si existe una ruta para "/usuarios"
4. Si existe, ejecuta el Controlador asociado a esa ruta
5. El Controlador usa el Modelo para obtener datos de la BD
6. El Controlador pasa los datos a una Vista
7. La Vista genera HTML
8. Laravel devuelve ese HTML al navegador del usuario

¿Qué es Artisan?

Artisan es la interfaz de línea de comandos (CLI) de Laravel. Es una de las herramientas más poderosas y se deberá usar constantemente.

¿Para qué sirve?

- Generar código automáticamente (controladores, modelos, migraciones).
- Ejecutar tareas como migrar la base de datos.
- Limpiar caché.
- Crear nuevos archivos con estructuras predefinidas.

Comandos básicos de Artisan:

Comando	Descripción
<i>php artisan list</i>	Muestra TODOS los comandos disponibles.
<i>php artisan help <comando></i>	Muestra ayuda sobre un comando específico.
<i>php artisan make:controller</i>	Crea un nuevo controlador.
<i>php artisan make:model</i>	Crea un nuevo modelo.
<i>php artisan make:migration</i>	Crea una nueva migración.
<i>php artisan migrate</i>	Ejecuta las migraciones en la BD.
<i>php artisan cache:clear</i>	Limpia la caché de la aplicación.

Actividad: seguir los pasos descritos para entender mejor el concepto.

Paso	Comando	Explicación
1	cd ~/proyectos/sge	Ir a la carpeta del proyecto.
2	php artisan list	Muestra todos los comandos disponibles. Tomarse 2 minutos para leerlos.
3	php artisan help make:controller	Muestra la ayuda del comando make:controller.
4	php artisan make:controller PruebaController	Crea un controlador llamado PruebaController.
5	ls app/Http/Controllers/	Verifica que el archivo PruebaController.php se creó.

Introducción a Laravel Sail

¿Qué es Laravel Sail?

Sail es una herramienta ligera que proporciona un entorno de desarrollo con Docker para Laravel. Básicamente, Sail usa Docker para crear contenedores con:

- PHP
- MySQL (o PostgreSQL)
- Redis (para caché)
- Nginx o Apache

¿Por qué usar Sail?

- No se necesita instalar PHP, MySQL, o Redis en su computadora.
- Todo corre dentro de contenedores Docker.
- El entorno es **reproducible**: funciona igual en cualquier computadora.
- Es compatible con Windows (WSL2), macOS y Linux.

¿Qué se necesita para usar Sail?

- Docker Desktop instalado y funcionando.
- WSL2 configurado.

Paso a paso para instalar Sail en el proyecto existente

Paso	Comando	Explicación
1	<code>cd ~/proyectos/sge</code>	Accede a la carpeta del proyecto.
2	<code>composer require laravel/sail --dev</code>	Instala Sail como dependencia de desarrollo.
3	<code>php artisan sail:install</code>	Ejecuta el instalador de Sail.
4	Elige los servicios: Seleccionar mysql (con las flechas y espacio).	Puede elegir Redis, Mailhog, etc. Por ahora solo MySQL.
5	Esperar a que termine la instalación.	Sail generará el archivo <code>compose.yml</code> en la raíz del proyecto.
6	<code>ls -la</code>	Verifica que aparezca el archivo <code>compose.yml</code> y la carpeta <code>vendor/</code> se ha actualizado.

Publicar Sail y revisar compose.yml

Explicación: al ejecutar `php artisan sail:install`, Sail creó un archivo docker-compose.yml en la raíz de su proyecto.

¿Qué contiene ese archivo? Define los contenedores Docker que se van a levantar:

- laravel.test: El contenedor principal con PHP y Nginx/Apache.
- mysql: El contenedor de la base de datos.
- redis (opcional): Para caché y sesiones.
- mailhog (opcional): Para probar envío de emails en local.

Línea por línea del docker-compose.yml (simplificado):

```
services:
  laravel.test:          # Servicio principal (PHP + Nginx)
    build:
      context: ./vendor/laravel/sail/runtimes/8.3
      dockerfile: Dockerfile
    ports:
      - "${APP_PORT:-80}:80"    # Mapea el puerto 80 del contenedor al puerto de su PC
    environment:
      - "DB_HOST=mysql"        # El host de la BD es "mysql" (nombre del servicio)
    depends_on:
      - mysql                  # Espera a que mysql esté listo

  mysql:                 # Servicio de base de datos
    image: 'mysql/mysql-server:8.0'
    ports:
      - "${FORWARD_DB_PORT:-3306}:3306"
    environment:
      MYSQL_ROOT_PASSWORD: '${DB_PASSWORD}'
      MYSQL_DATABASE: '${DB_DATABASE}'
    volumes:
      - 'sail-mysql:/var/lib/mysql'  # Volumen persistente
```

¿Qué es un volumen persistente? Los datos de la BD se guardan en un volumen Docker. Así, aunque se apague el contenedor, los datos no se pierden.

Levantar Sail y verificar

¿Cómo se levanta Sail?

Se debe ejecutar el comando: `./vendor/bin/sail up -d`

- sail up: Levanta los contenedores.
- -d: Modo "detached" (en segundo plano). Así la terminal queda libre.
- **¿Qué pasa si solo se escribe sail up sin -d?** se mostrarán los logs en tiempo real (útil para depurar errores).

¿Cómo se detiene Sail?

Se debe ejecutar el comando: `./vendor/bin/sail down`

Paso a paso para levantar Sail:

Paso	Comando	Explicación
1	<code>cd ~/proyectos/sge</code>	Ve a la carpeta del proyecto.
2	<code>./vendor/bin/sail up -d</code>	Levanta los contenedores en segundo plano. La primera vez puede tomar varios minutos (descarga imágenes de Docker).
3	<code>docker ps</code>	Muestra los contenedores en ejecución. Debería verse sge-laravel-laravel.test-1, sge-laravel-mysql-1, etc.
4	Abrir el navegador en http://localhost	Nota: Sail usa el puerto 80 por defecto, NO el 8000. Se mostrará la pantalla de bienvenida de Laravel.

Si el puerto 80 está ocupado: modificar la variable APP_PORT en el archivo .env (ej: APP_PORT=8080) y reinicia Sail con:

```
./vendor/bin/sail down
```

```
./vendor/bin/sail up -d.
```

Extensión (Error de Sesiones)

¿Por qué ocurre este error?

Cuando se accede a `http://localhost`, Laravel intenta leer o escribir una sesión para identificar al visitante. Por defecto, Laravel almacena las sesiones en la base de datos (en la tabla `sessions`). Pero como **no se ha ejecutado las migraciones**, esa tabla no existe.

Illuminate\Database\QueryException

vendor/laravel/framework/src/Illuminate/Database/Connection.php:843

SQLSTATE[HY000] [2002] php_network_getaddresses: getaddrinfo for mysql failed: Name or service not known (Connection: mysql, Host: mysql, Port: 3307, Database: seminario_db, SQL: select * from `sessions` where `id` = WxT6Q6CHpHyxtgSLt4nh13axWtv00KThDdxyfxxP limit 1)

¿Qué son las migraciones?

Son archivos PHP que definen la estructura de las tablas de la base de datos. Laravel incluye migraciones predefinidas para tablas como:

- users (usuarios)
- password_reset_tokens (restablecimiento de contraseñas)
- failed_jobs (tareas fallidas)
- sessions (sesiones)
- cache (caché)

Estas migraciones ya vienen con Laravel, pero **no se han ejecutado** en su base de datos. Por eso el error.

El archivo .env (environment)

El archivo `.env` (environment) contiene las **variables de entorno** de su aplicación. Estas son configuraciones que cambian según el entorno (local, producción, etc.).

¿Por qué es importante?

- Separa la configuración del código.
- Permite tener configuraciones diferentes en local y en producción.
- **Nunca se sube a GitHub** (está en `.gitignore`).

Variables principales en .env:

Variable	¿Qué configura?	Ejemplo
APP_NAME	Nombre de la aplicación	"SGE"
APP_ENV	Entorno (local, production)	local
APP_DEBUG	Modo depuración (true/false)	true
APP_URL	URL de la aplicación	http://localhost
DB_CONNECTION	Motor de BD	mysql
DB_HOST	Host de la BD	mysql (nombre del servicio Sail)
DB_PORT	Puerto de la BD	3306
DB_DATABASE	Nombre de la BD	laravel
DB_USERNAME	Usuario de la BD	sail
DB_PASSWORD	Contraseña de la BD	password

Importante: Cuando se usa Sail, el DB_HOST debe ser mysql (el nombre del servicio en el docker-compose.yml). No se debe usar localhost o 127.0.0.1 porque la BD está dentro de un contenedor Docker.

Paso a paso para solucionarlo

Paso 1: Verificar la conexión a la base de datos

Antes de ejecutar migraciones, hay que asegurarse de que la conexión a la base de datos es correcta. Abrir el archivo `.env` y verificar estos valores:

```
DB_CONNECTION=mysql
DB_HOST=mysql
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=sail
DB_PASSWORD=password
```

Nota: Asegurarse de que `DB_HOST` sea `mysql` (el nombre del servicio en Docker). No se debe poner `localhost` porque la BD está dentro de un contenedor.

Paso 2: Ejecutar las migraciones

Laravel tiene un comando Artisan que ejecuta TODAS las migraciones pendientes:

```
./vendor/bin/sail php artisan migrate
```

¿Qué hace este comando?

- Busca todos los archivos de migración en `database/migrations/`.
- Los ejecuta en orden (por fecha) sobre la base de datos.
- Crea las tablas necesarias: `users`, `sessions`, `failed_jobs`, `password_reset_tokens`, etc.

Salida esperada:

Migration table created successfully.

Migrating: 2014_10_12_000000_create_users_table

Migrated: 2014_10_12_000000_create_users_table (0.02 seconds)

Migrating: 2014_10_12_100000_create_password_reset_tokens_table

Migrated: 2014_10_12_100000_create_password_reset_tokens_table (0.01 seconds)

Migrating: 2019_08_19_000000_create_failed_jobs_table

Migrated: 2019_08_19_000000_create_failed_jobs_table (0.01 seconds)

Migrating: 2019_12_14_000001_create_personal_access_tokens_table

Migrated: 2019_12_14_000001_create_personal_access_tokens_table (0.02 seconds)

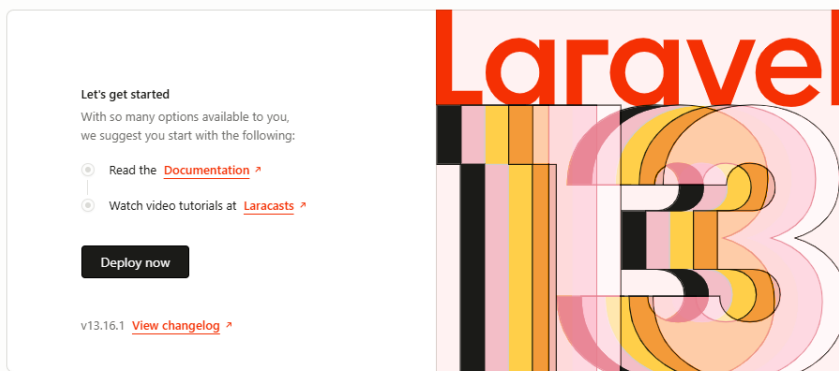
Migrating: 2024_..._..._..._..._..._..._create_sessions_table

Migrated: 2024_..._..._..._..._..._..._create_sessions_table (0.01 seconds)

Si se muestra el mensaje anterior, indica que todo está bien.

Paso 3: Recargar la página

Ahora se debe acceder a <http://localhost> y recarga la página. **El error debería desaparecer** y se mostrará la pantalla de bienvenida de Laravel.



¿Qué pasó exactamente?

Cuando se ejecutó migrate, Laravel:

1. Creó la tabla migrations (para llevar el control de qué migraciones ya se ejecutaron).
2. Ejecutó cada archivo de migración en orden.
3. Creó la tabla sessions (entre otras).
4. Ahora, cuando Laravel intenta leer/escribir una sesión, la tabla ya existe y funciona sin errores.

Errores comunes:

1. **Puerto 80 ocupado:** Algunos pueden tener Skype, Teams o IIS usando el puerto 80. Se puede cambiar el APP_PORT en .env.
2. **Docker no inicia:** Verificar que Docker Desktop esté ejecutándose y que WSL2 esté habilitado en la configuración.
3. **Composer no se encuentra:** Asegurarse de que Composer esté instalado globalmente (/usr/local/bin/composer).

Acceder a la base de datos desde la terminal (sin phpMyAdmin)**Sail incluye un atajo para conectarse a MySQL:**

```
./vendor/bin/sail mysql
```

Si lo anterior no funciona, se puede ejecutar: `docker exec -it sge-laravel.test-1 bash`

Y luego `mysql -h mysql -u sail -p`

Esto abre el cliente MySQL dentro del contenedor, usando automáticamente las credenciales del .env. Dentro del cliente, ejecutar:

```
SHOW DATABASES;  
USE laravel;  
SHOW TABLES;
```

Salida esperada: La base de datos “laravel” existe con las tablas requeridas si se ejecutó la migración con anterioridad.

Ejecutar las migraciones (crear las tablas) – Si no se han ejecutado con anterioridad

```
./vendor/bin/sail php artisan migrate
```

Salida esperada: Se ejecutan todas las migraciones y se crean las tablas users, sessions, failed_jobs, etc.

Verificar las tablas desde la terminal

```
./vendor/bin/sail mysql
```

Dentro de MySQL:

```
USE laravel;  
SHOW TABLES;
```

Ahora se deben ver las tablas creadas. Para salir de la consola de MySQL, se ejecuta el comando:

```
exit
```

¿Cómo ver la base de datos de forma gráfica sin phpMyAdmin?

Usar un cliente gráfico externo (recomendado)

Se puede usar **DBeaver**, **HeidiSQL**, **TablePlus**, **MySQL Workbench** o cualquier cliente MySQL. Configurar la conexión con:

- **Host:** 127.0.0.1 (o localhost)
- **Puerto:** 3306 (o el que se haya definido en DB_PORT)
- **Usuario:** sail
- **Contraseña:** password
- **Base de datos:** laravel

Primera modificación: Vista Welcome

¿Dónde está la pantalla de bienvenida de Laravel?

La pantalla que se muestra en `http://localhost` es una vista llamada `welcome.blade.php` que está en `resources/views/`.

¿Qué es Blade? Es el motor de plantillas de Laravel. Permite escribir HTML con PHP embebido de forma muy limpia. Se explorará en detalle la próxima semana.

Por ahora, solo se modificará el texto de bienvenida. Paso a paso para modificar la vista Welcome:

Paso	Acción	Explicación
1	Abrir <code>resources/views/welcome.blade.php</code> en VS Code.	Este archivo contiene el HTML de la página de bienvenida.
2	Busca el texto Laravel (puede estar dentro de un <code><h1></code> o un <code><title></code>).	
3	Cambiar el título de la página a "Bienvenidos al Seminario Laravel".	Buscar <code><title></code> y cambiarlo.
4	Cambiar el mensaje principal a "Este es nuestro primer proyecto con Laravel".	Buscar el <code><h1></code> o el texto principal y cambiarlo.
5	Guardar el archivo (Ctrl + S).	
6	Recargar la página en <code>http://localhost</code> .	Se deben ver los cambios reflejados.

Nota: Como el código está montado con Docker (volumen), los cambios se reflejan instantáneamente. No se necesita reiniciar el servidor.

ENTREGABLE DE LA SEMANA

Cada grupo debe subir a su repositorio de GitHub:

- **Directorio** con TODAS las capturas de pantalla: Captura 1: php -v y composer -version.
 - Captura 2: ls -la de la estructura del proyecto.
 - Captura 3: Pantalla de bienvenida de Laravel (sin Sail).
 - Captura 4: php artisan list mostrando comandos.
 - Captura 5: docker ps mostrando contenedores de Sail.
 - Captura 6: Archivo .env modificado.
 - Captura 7: Página de bienvenida modificada.
- **Archivo** README.md actualizado con: Capítulo 2: Instalación de Laravel.
 - Explicación de la estructura de carpetas.
 - Diagrama del flujo de una petición.
 - Explicación de las variables de entorno.
- **El código del proyecto** (sin vendor/ ni .env).

Comandos para subir a GitHub:

bash

git add .

git commit -m "Semana 2: Instalación Laravel + Sail - Estructura y MVC"

git push